

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

**B. Tech. IV Year I semester
Course Code: 8EC08**

BY

B. VASUNDHARA DEVI
ASSISTANT PROFESSOR

What is AI?

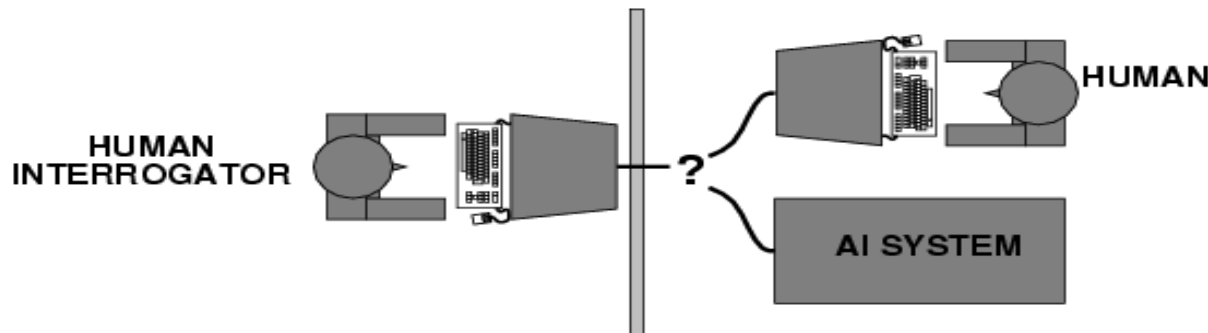
“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ...” (Bellman, 1978)	“The study of mental faculties through the use of computational models” (Charniak+McDermott, 1985)
“The study of how to make computers do things at which, at the moment, people are better” (Rich+Knight, 1991)	“The branch of computer science that is concerned with the automation of intelligent behavior” (Luger+Stubblefield, 1993)

Views of AI fall into four categories:

Thinking humanly	Thinking rationally
Acting humanly	Acting rationally

Acting humanly: Turing Test

- Turing (1950) "Computing machinery and intelligence":
- "Can machines think?" → "Can machines behave intelligently?"
- Operational test for intelligent behavior: the Imitation Game
- A Computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.



Thinking humanly: cognitive modeling

- 1960s "cognitive revolution": information-processing psychology Requires scientific theories of internal activities of the brain
- How to validate? Requires
 - 1) Predicting and testing behavior of human subjects (top-down)
 - or
 - 2) Direct identification from neurological data (bottom-up)
- The interdisciplinary field of cognitive science brings together computer models from AI and experimental techniques from psychology to construct precise and testable theories of the human mind.

Thinking rationally: "laws of thought"

- Syllogisms provided patterns for argument structures that always yielded correct conclusions when given correct premises.
- Ex: “Socrates is a man; All men are mortal; therefore, Socrates is mortal.” These laws of thought were supposed to govern the operation of the mind; their study initiated the field called logic
- Problems:
 1. Not all intelligent behavior is mediated by logical deliberation
 2. What is the purpose of thinking? What thoughts should I have?

Acting rationally: rational agent

- An agent is just something that acts (agent comes from the Latin agere, to do).
- A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.
- Rational behavior: doing the right thing
- The right thing: that which is expected to maximize goal achievement, given the available information
- Knowledge representation and Reasoning enable Agents to reach good decisions.
- “laws of thought” approach to AI, emphasis was on correct inferences.

Rational Agents

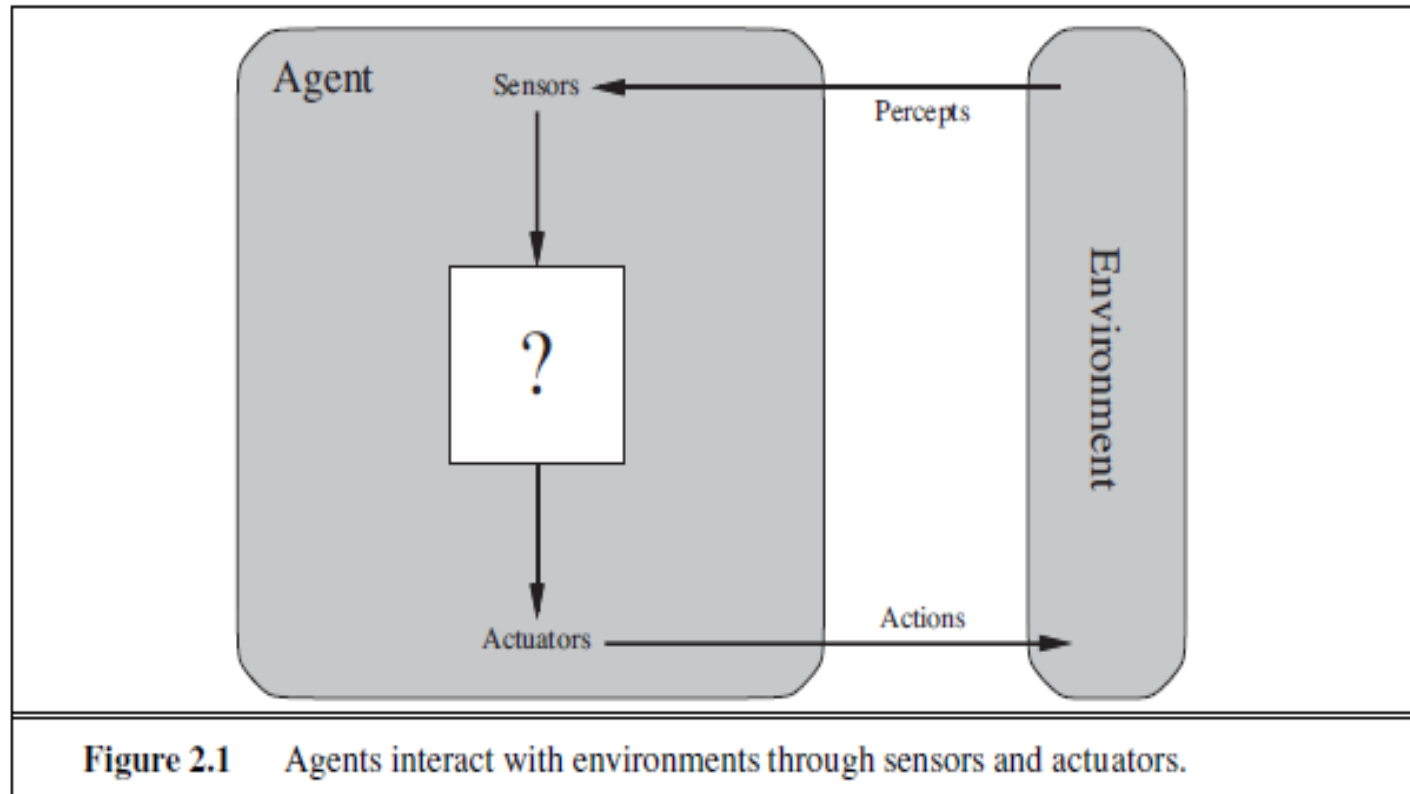
- An agent is an entity that perceives and acts
- This course is about designing rational agents
- Abstractly, an agent is a function from percept histories to actions:

$$[f: \mathcal{P}^* \rightarrow \mathcal{A}]$$

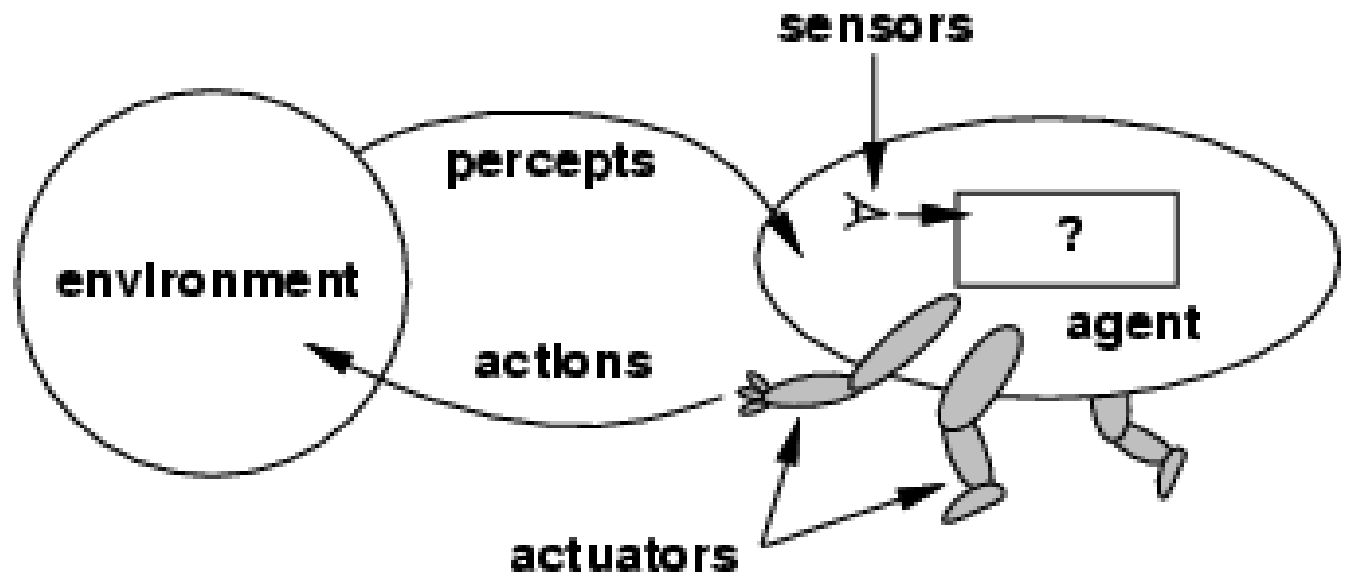
- For any given class of environments and tasks, we seek the agent (or class of agents) with the best performance
- Caveat: computational limitations make perfect rationality unachievable
 - design best program for given machine resources

INTELLIGENT AGENTS

- AGENTS AND ENVIRONMENTS
- An Agent is anything that can be ENVIRONMENT viewed as perceiving its environment through sensors and SENSOR acting upon that environment through actuators.

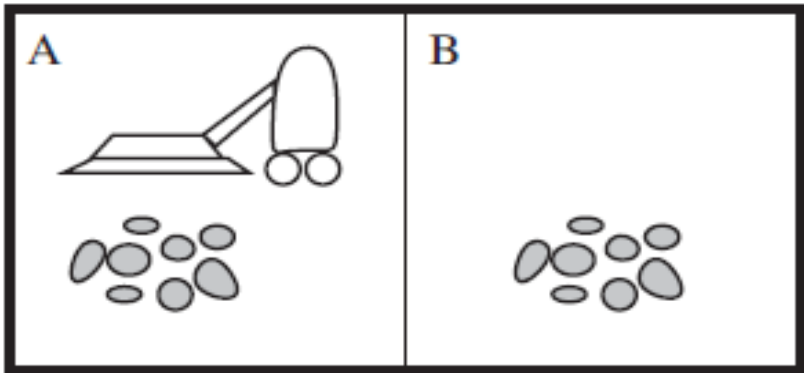


- A Human Agent has eyes, ears, and other organs for sensors and hands, legs, vocal tract, and so on for actuators.
- A Robotic Agent might have cameras and infrared range finders for sensors and various motors for actuators.
- Agent's behavior is described by the **agent function** that maps any given percept (interpret) sequence to an action



- Tabulating the Agent Function describes any given agent, construct this table by trying out all possible percept sequences and recording which actions the agent does in response.
- Table is an external characterization of the agent and Internally, agent function for an artificial agent will be implemented by an agent program.
- Agent function is an abstract mathematical description
- Ex: vacuum-cleaner Agent world shown in Figure 2.2. This world is so simple that we can describe everything that happens; world has just two locations: squares A and B.

Figure 2.2 A vacuum-cleaner world with just two locations.



Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

GOOD BEHAVIOR: THE CONCEPT OF RATIONALITY

- Rational Agent:
- It is one that does the right thing - conceptually speaking, every entry in the table for the agent function is filled out correctly.
- An agent should strive to "do the right thing", based on what it can perceive and the actions it can perform.
- Performance measure: An objective criterion for success of an agent's behavior
- Ex: Performance measure of a vacuum-cleaner agent could be amount of dirt cleaned up, amount of time taken, amount of electricity consumed, amount of noise generated, etc.

RATIONALITY

What is rational at any given time depends on four things:

- The performance measure that defines the criterion of success.
- The agent's prior knowledge of the environment.
- The actions that the agent can perform.
- The agent's percept sequence to date.

Rational Agent: For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

- As a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

THE NATURE OF ENVIRONMENTS

- Think about task environments, which are essentially the “problems” to which rational agents are the “solutions.”

PEAS (PERFORMANCE, ENVIRONMENT, ACTUATORS, SENSORS) DESCRIPTION

- The vacuum world was a simple example; let us consider a more complex problem: an automated taxi driver.
- The full driving task is extremely *open-ended*.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figure 2.4 PEAS description of the task environment for an automated taxi.

THE STRUCTURE OF AGENTS

- The job of AI is to design an agent program that implements the agent function— the mapping from percepts to actions. We assume this program will run on some sort of computing device with physical sensors and actuators—

agent = architecture + program .

- If the program is going to recommend actions like *Walk*, the architecture had better have legs.
- Architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the actuators as they are generated.

Agent programs

- Agent programs take the current percept as input from the sensors and return an action to the actuators.
- Difference between the agent program, which takes the current percept as input, and the agent function, which takes the entire percept history.
- The agent program takes just the current percept as input because nothing more is available from the environment;
- if the agent's actions need to depend on the entire percept sequence, the agent will have to remember the percepts.

- For example, Figure 2.7 shows a rather trivial agent program that keeps track of the percept sequence and then uses it to index into a table of actions to decide what to do.
- To build a rational agent in this way, as a designers must construct a table that contains the appropriate action for every possible percept sequence.

- The daunting size of these tables means that
 - (a) no physical agent in this universe will have the space to store the table
 - (b) the designer would not have time to create the table
 - (c) no agent could ever learn all the right table entries from its experience, and
 - (d) even if the environment is simple enough to yield a feasible table size, the designer still has no guidance about how to fill in the table entries

For ex: Huge tables of square roots used by engineers and schoolchildren prior to the 1970s have now been replaced by a five-line program.

- Four basic kinds of agent programs that embody the principles underlying almost all intelligent systems:
 - Simple reflex agents;
 - Model-based reflex agents;
 - Goal-based agents; and
 - Utility-based agents
- Each kind of agent program combines particular components in particular ways to generate actions
- Explains in general terms how to convert all these agents into *learning agents that can improve the performance of their components so as to generate better actions.*

Simple reflex agents

Ex: the vacuum agent whose agent function is tabulated in Figure 2.3 is a simple reflex agent, because its decision is based only on the current location and on whether that location contains dirt.

An agent program for this agent is shown in Figure 2.8.

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
```

```
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

Figure 2.8 The agent program for a simple reflex agent in the two-state vacuum environment. This program implements the agent function tabulated in Figure 2.3.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
```

```
  persistent: rules, a set of condition-action rules
```

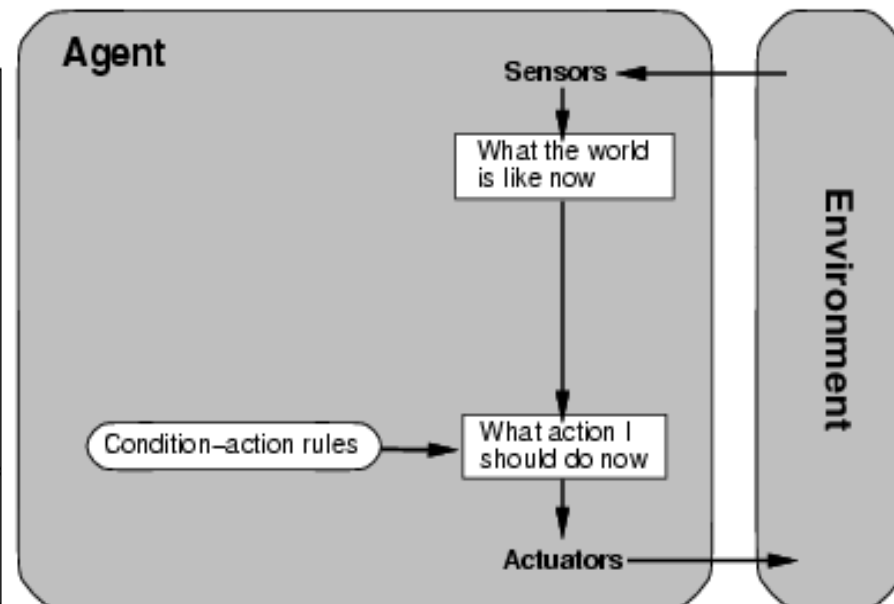
```
  state ← INTERPRET-INPUT(percept)
```

```
  rule ← RULE-MATCH(state, rules)
```

```
  action ← rule.ACTION
```

```
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.



- Simple reflex behaviors occur even in more complex environments.
- Imagine yourself as the driver of the automated taxi.
- If the car in front brakes and its brake lights come on, then you should notice this and initiate braking.
- Triggers some established connection in the agent program to the action “initiate braking.”
- Call such a connection a condition–action rule, written as
if car-in-front-is-braking then initiate-braking
- The INTERPRET-INPUT function generates an abstracted description of the current state from the percept, and the RULE-MATCH function returns the first rule in the set of rules that matches the given state description.

- The agent in Fig 2.10 will work *only if the correct decision can be made on the basis of only the current percept—that is, only if the environment is fully observable.*
- This works if the car in front has a centrally mounted brake light.
- A simple reflex agent driving behind such a car would either brake continuously and unnecessarily, or, worse, never brake at all.
- Infinite loops are often unavoidable for simple reflex agents operating in partially observable environments.
- Ex: if the vacuum agent perceives [Clean], it might flip a coin to choose between Left and Right .

Model-based reflex agents

- The most effective way to handle partial observability is for the agent to *keep track of the part of the world it can't see now*.
- *Agent should maintain some sort of internal* state that depends on the percept history and thereby reflects at least some of the unobserved aspects of the current state.
- For the braking problem, the internal state is not too extensive—just the previous frame from the camera, allowing the agent to detect when two red lights at the edge of the vehicle go on or off simultaneously.
- Internal state information requires two kinds of knowledge to be encoded in the agent program.
- Need some information about how the world evolves independently of the agent—for example, that an overtaking car generally will be closer behind than it was a moment ago.

- Second, need some information about how the agent's own actions affect the world
- Ex: when the agent turns the steering wheel clockwise, the car turns to the right, or that after driving for five minutes northbound on the freeway, one is usually about five miles north of where one was five minutes ago.
- This knowledge about “how the world works”—whether implemented in simple Boolean circuits or in complete scientific theories—is called a model of the world.
- An agent that uses such a model is called a model-based agent.

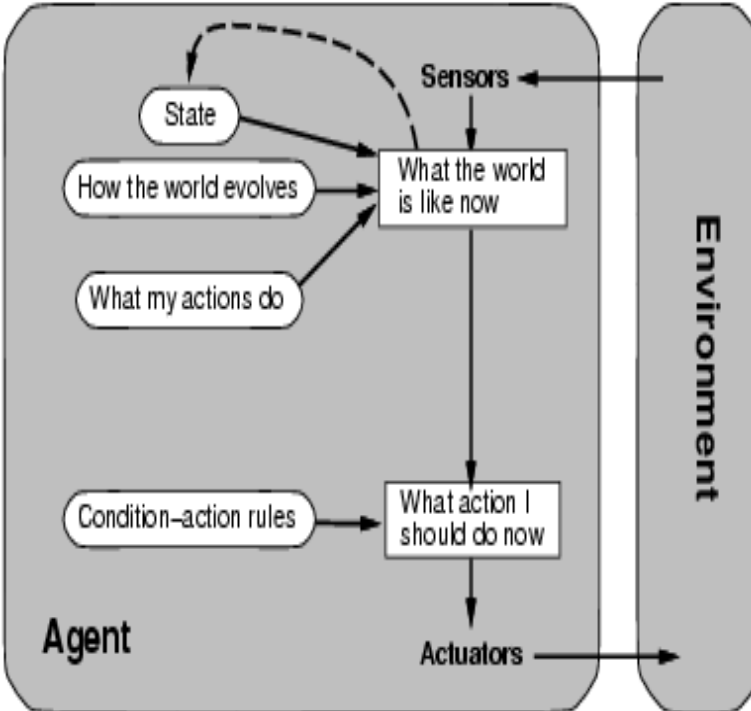


Figure 2.11 A model-based reflex agent

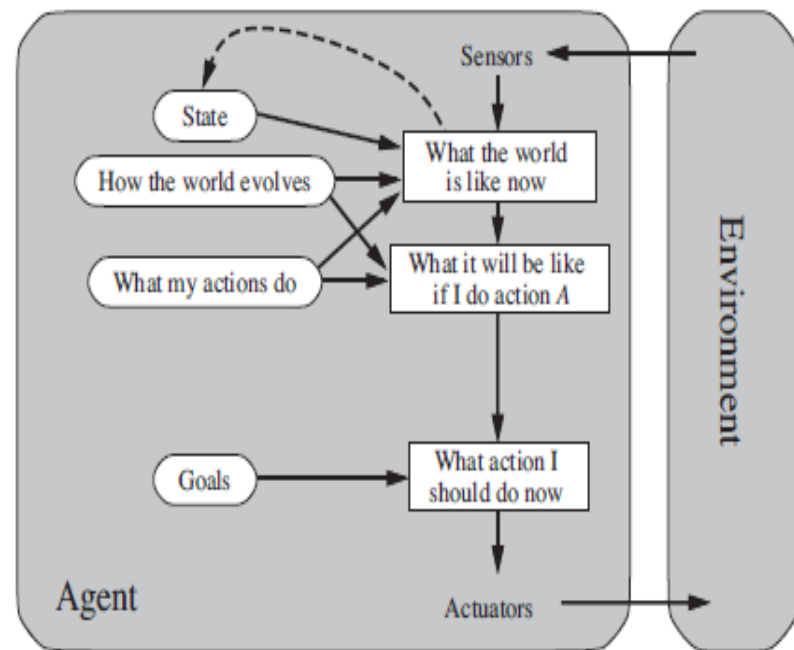


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

persistent: *state*, the agent's current conception of the world state

model, a description of how the next state depends on current state and action

rules, a set of condition-action rules

action, the most recent action, initially none

state ← UPDATE-STATE(*state*, *action*, *percept*, *model*)

rule ← RULE-MATCH(*state*, *rules*)

action ← *rule*.ACTION

return *action*

Figure 2.12 A model-based reflex agent. It keeps track of the current state of the world, using an internal model. It then chooses an action in the same way as the reflex agent.

- The details of how models and states are represented vary widely depending on the type of environment.
- It is possible for the agent to determine the current state of a partially observable environment *exactly*.
- *Box* labeled “what the world is like now” represents the agent’s “best guess” (or sometimes best guesses).
- Ex: An automated taxi may not be able to see around the large truck that has stopped in front of it and can only guess about what may be causing the hold-up.

Goal-based agents

- Knowing something about the current state of the environment is not always enough to decide what to do.
- Ex: At a road junction, the taxi can turn left, turn right, or go straight on.
- The correct decision depends on where the taxi is trying to get to.
- Current state description, the agent needs some sort of goal information that describes situations that are desirable—for example, being at the passenger's destination.
- The agent program can combine this with the model (the same information as was used in the model based reflex agent) to choose actions that achieve the goal.
- Figure 2.13 shows the goal-based agent's structure.
- Goal-based action selection is straightforward,
- Ex: when goal satisfaction results immediately from a single action.

- Sometimes it will be more tricky
- Ex: when the agent has to consider long sequences of twists and turns in order to find a way to achieve the goal.
- **Search** and **planning** are the subfields of AI devoted to finding action sequences that achieve the agent's goals.
- In the reflex agent designs, this information is not explicitly represented, because the built-in rules map directly from percepts to actions.
- The reflex agent brakes when it sees brake lights.
- A goal-based agent, reason that if the car in front has its brake lights on, it will slow down.

Utility-based agents

- Goals alone are not enough to generate high-quality behavior in most environments.
- Ex: Many action sequences will get the taxi to its destination (thereby achieving the goal) but some are quicker, safer, more reliable, or cheaper than others.
- Goals just provide a crude binary distinction between “happy” and “unhappy” states.
- A more general performance measure should allow a comparison of different world states according to exactly how happy they would make the agent.
- Because “happy” does not sound very scientific, economists and computer scientists use the term utility instead
- Performance measure assigns a score to any given sequence of environment states, so it can easily distinguish between more and less desirable ways of getting to the taxi’s destination.

- An agent's **utility function** is **essentially an internalization** of the performance measure.
- It has many advantages in terms of flexibility and learning.
- First, when there are conflicting goals, only some of which can be achieved (ex: speed and safety), the utility function specifies the appropriate tradeoff.
- Second, when there are several goals that the agent can aim for, none of which can be achieved with certainty, utility provides a way in which the likelihood of success can be weighed against the importance of the goals.
- A rational utility-based agent chooses the action that maximizes the expected utility of the action outcomes.

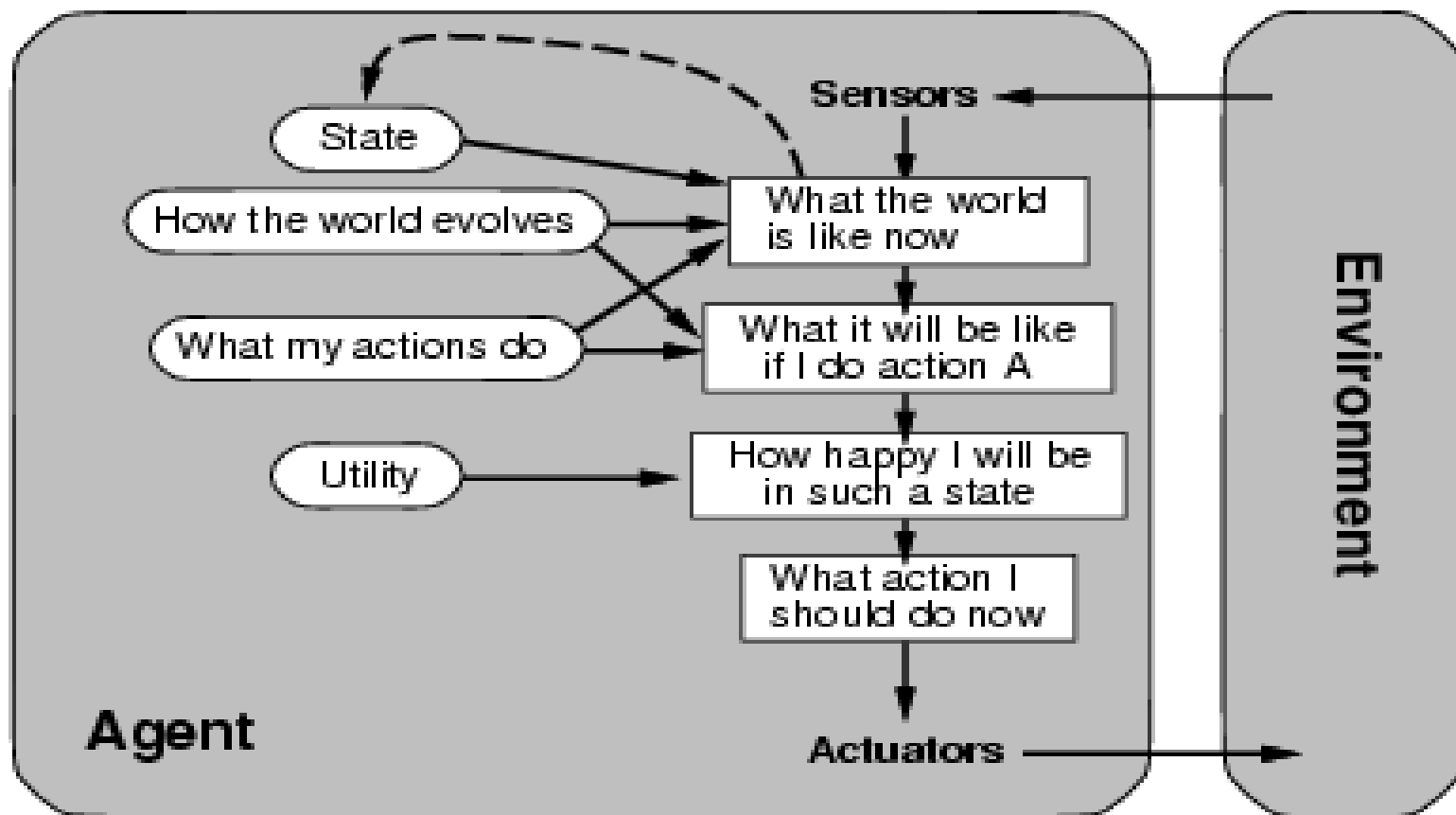


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along **with** a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Learning agents

- Described agent programs with various methods for selecting actions but not explained how the agent programs *come into being*.
- Learning has another advantage: it allows the agent to operate in initially unknown environments and to become more competent than its initial knowledge alone might allow.
- A learning agent can be divided into four conceptual components, as shown in Fig. 2.15.
- The most important distinction is between the learning element, which is responsible for making improvements, and the **performance element, which is responsible for** selecting external actions.
- The performance element is considered to be the entire agent: it takes in percepts and decides on actions.

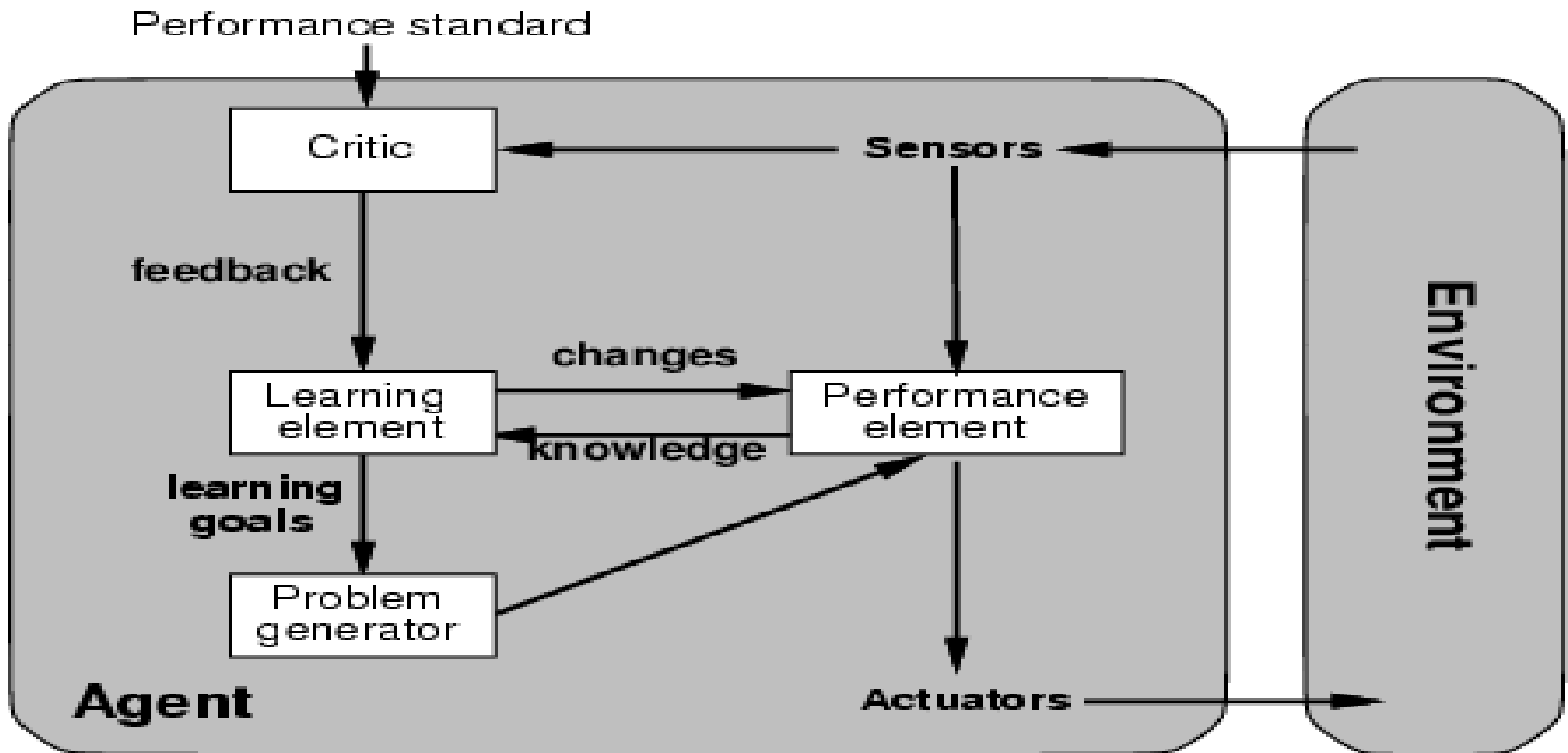


Figure 2.15 A general learning agent.

The learning element uses feedback from the critic on how the agent is doing and determines how the performance element should be modified to do better in the future.

The design of the learning element depends very much on the design of the performance element.

- Given an agent design, learning mechanisms can be constructed to improve every part of the agent.
- Ex: A chess program could receive a percept indicating that it has checkmated its opponent, but it needs a performance standard to know that this is a good thing.
- The last component of the learning agent is the **problem generator**.
- The point is that if the performance element had its way, it would keep doing the actions that are best, given what it knows.
- But if the agent is willing to explore a little and do some perhaps suboptimal actions in the short run, it might discover much better actions for the long run.
- The performance element consists of whatever collection of knowledge and procedures the taxi has for selecting its driving actions.
- Ex: After the taxi makes a quick left turn across three lanes of traffic, the critic observes the shocking language used by other drivers.

- From this experience, the learning element is able to formulate a rule saying this was a bad action, and the performance element is modified by installation of the new rule.
- The learning element can make changes to any of the “knowledge” components shown in the agent diagrams.
- Ex: If the taxi exerts a certain braking pressure when driving on a wet road, then it will soon find out how much deceleration is actually achieved.
- The external performance standard must inform the agent that the loss of tips is a negative contribution to its overall performance; then the agent might be able to learn that violent maneuvers do not contribute to its own utility.

How the components of agent programs work

- Describe Agent programs (in very high-level terms) as consisting of various components, whose function it is to answer questions such as: “What is the world like now?” “What action should I do now?” “What do my actions do?”
- Place the representations along an axis of increasing complexity and expressive power—**atomic**, **factored**, and **structured**

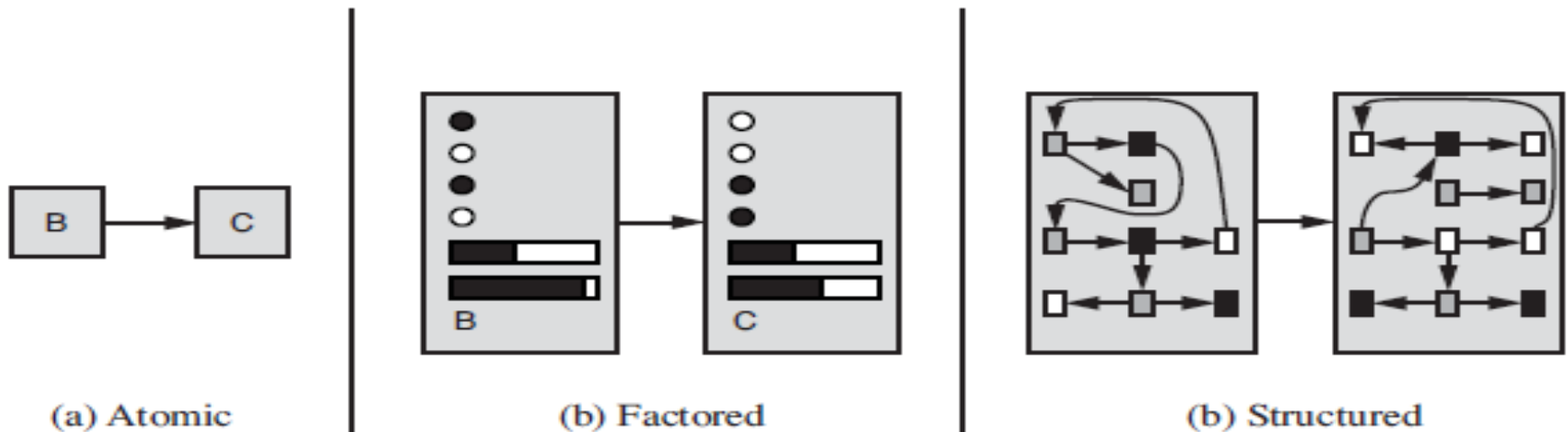


Figure 2.16 Three ways to represent states and the transitions between them. (a) Atomic representation: a state (such as B or C) is a black box with no internal structure; (b) Factored representation: a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols. (c) Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Problem solving agents

- **problem-solving agent** is one kind of goal-based agent.
- Problem-solving agents use **atomic representations**
- Goal-based agents that use more advanced **factored or structured representations** are usually called **planning agents**
- General-purpose search algorithms that can be used to solve these problems
- **Uninformed search algorithms**—algorithms that are given **no information about the problem** other than its definition.
- some of these algorithms can solve any solvable problem, none of them can do so efficiently.
- **Informed search algorithms, on the other hand**, can do quite well given some guidance on where to look for solutions.
- *More general case*—where the agent's future actions may vary depending on future percepts.
- It uses the concepts of asymptotic complexity (that is, $O()$ notation) and NP-completeness.

PROBLEM-SOLVING AGENTS

- Intelligent agents are supposed to maximize their performance measure.
- Imagine an agent in the city of Arad, Romania, enjoying a touring holiday.
- The agent's performance measure contains many factors: it wants to improve its suntan, improve its Romanian, take in the sights, enjoy the nightlife (such as it is), avoid hangovers, and so on.
- Now, suppose the agent has a nonrefundable ticket to fly out of Bucharest the following day.
- In that case, it makes sense for the agent to adopt the **goal of getting to Bucharest**.
- Goals help organize behavior by limiting the objectives that the agent is trying to achieve.
- **Goal formulation, based on the current situation and the agent's performance** measure, is the first step in problem solving.
- If it were to consider actions at the level of "move the left foot forward an inch" or "turn the steering wheel one degree left," the agent would probably never find its way out of the parking lot, let alone to Bucharest.

- **Problem formulation** is the process of deciding what actions and states to consider, given a goal.
- Assume that the agent will consider actions at the level of driving from one major town to another and Each State represents a town
- Our agent has now adopted the goal of driving to Bucharest and is considering where to go from Arad.
- Three roads lead out of Arad, one toward Sibiu, one to Timisoara, and one to Zerind.
- The agent can use this information to consider *subsequent stages of a hypothetical journey via* each of the three towns, trying to find a journey that eventually gets to Bucharest.
- Once it has found a path on the map from Arad to Bucharest, it can achieve its goal by carrying out the driving actions.

- Assume that the environment is **observable**, so the agent always knows the current state.
- Assume the environment is **discrete**, so at any given state there are only finitely many actions to choose from.
- This is true for navigating in Romania because each city is connected to a small number of other cities.
- Assume that the environment is **deterministic**, so each action has **exactly one** outcome. Under ideal conditions, this is true for the agent in Romania—it means that if it chooses to drive from Arad to Sibiu, it does end up in Sibiu.
- Ex: Under less than ideal conditions, the agent might plan to drive from Arad to Sibiu and then to Rimnicu Vilcea but may also need to have a contingency plan in case it arrives by accident in Zerind instead of Sibiu.

- Since only one percept is possible after the first action, the solution can specify only one possible second action, and so on.
- The process SEARCH of looking for a sequence of actions that reaches the goal is called **search**.
- A search algorithm takes a problem as input and returns a **solution in the form of an action** sequence.
- Once a solution is found, the actions it recommends can be carried out.
- This is called the **execution phase**.
- Thus, we have a simple “**formulate, search, execute**” design for the agent, as shown in Figure 3.1.
- After formulating a goal and a problem to solve, the agent calls a search procedure to solve it.
- An agent that carries out its plans with its eyes closed, so to speak, must be quite certain of what is going OPEN-LOOP on.

Problem-solving agents

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

Well-defined problems and solutions

- **Problem can be defined formally by five components:**
- The **initial state** that the agent starts in.
Ex: The initial state for our agent in Romania might be described as $\text{In}(\text{Arad})$.
- A **description** of the possible actions available to the agent.
Given a particular state s , $\text{ACTIONS}(s)$ returns the set of actions that can be executed in s .
Ex: from the state $\text{In}(\text{Arad})$, the applicable actions are $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \text{Go}(\text{Zerind})\}$.
- **RESULT(s, a)** that returns the state that results from doing action a in state s .
- We also use the term **successor** to refer to any state reachable from a given state by a single action. Ex: we have

$$\text{RESULT}(\text{In}(\text{Arad}), \text{Go}(\text{Zerind})) = \text{In}(\text{Zerind})$$

- **State space** of the problem—is defined as the set of all states reachable from the initial state by any sequence of actions.
- A **path in the state space is a sequence** of states connected by a sequence of actions.
- The **goal test**, which determines whether a given state is a goal state.
- Agent's goal in Romania is the singleton set {In(Bucharest)}.
- A **path cost function** that assigns a numeric cost to each path.
- Cost of a path can be described as the *sum of the costs of the individual actions along the path*. The **step cost of taking action a** in state **s** to reach state **s'** is denoted by $c(s, a, s')$.
- The step costs for Romania are shown in Fig. 3.2 as route distances.
- A **solution to a problem is an action sequence** that leads from the initial state to a goal state. Solution quality is measured by the path cost function, and an **optimal solution has the lowest path cost among all solutions**.

EXAMPLE PROBLEMS

- The problem-solving approach has been applied to a vast array of task environments.
- List some of the best known here, distinguishing between *toy and real-world problems*.
- A **toy problem** is intended to illustrate or exercise various problem-solving methods.
- It can be given a concise, exact description and hence is usable by different researchers to compare the performance of algorithms.
- A **real-world problem** is one whose solutions people **actually** care about.
- Such problems tend not to have a single agreed-upon description, but we can give the general flavor of their formulations.

8-puzzle

- The **8-puzzle**, an instance of which is shown in Figure 3.4, consists of a 3×3 board with eight numbered tiles and a blank space.

A tile adjacent to the blank space can slide into the space.

The object is to reach a specified goal state, such as the one shown on the right of the figure.

The standard formulation is as follows:

- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states (Exercise 3.4).
- **Actions:** The simplest formulation defines the actions as movements of the blank space *Left, Right, Up, or Down*. *Different subsets of these are possible depending on where the blank is.*

- **Transition model:** Given a state and action, this returns the resulting state; for example, if we apply *Left to the start state in Figure 3.4*, the resulting state has the 5 and the blank switched.
- **Goal test:** This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

The 8-puzzle belongs to the family of sliding-block puzzles, which are often used as test problems for new search algorithms in AI.

This family is known to be NP-complete, so one does not expect to find methods significantly better in the worst case than the search algorithms.

- The 8-puzzle has $9!/2=181,440$ reachable states and is easily solved.
- The 15-puzzle (on a 4×4 board) has around 1.3 trillion states, and random instances can be solved optimally in a few milliseconds by the best search algorithms.
- The 24-puzzle (on a 5×5 board) has around 10^{25} states, and random instances take several hours to solve optimally

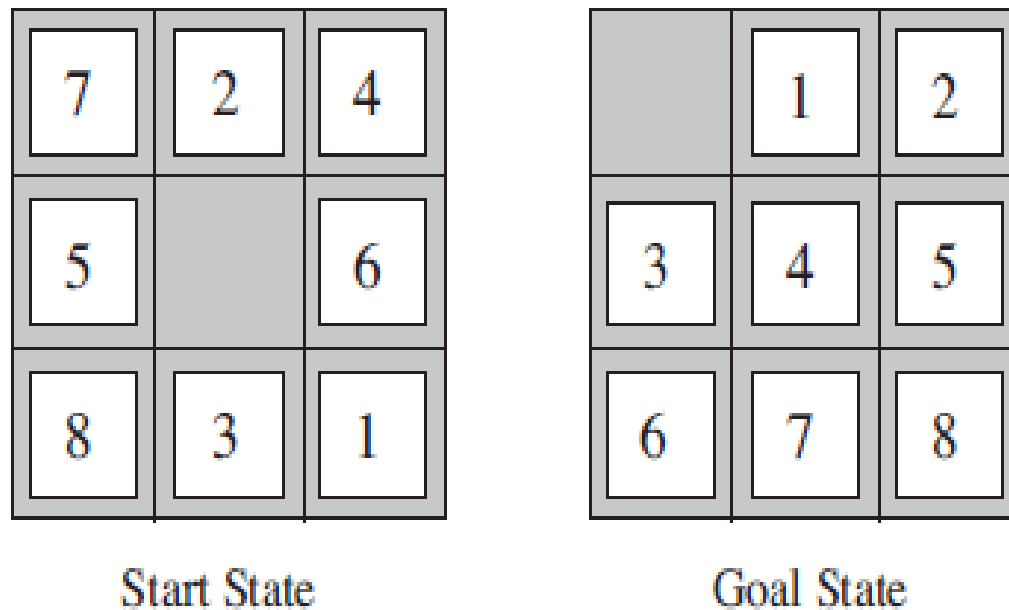


Figure 3.4 A typical instance of the 8-puzzle.

8-queens problem

- The goal of the **8-queens problem** is to place eight queens on a chessboard such that no queen attacks any other.
- (A queen attacks any piece in the same row, column or diagonal.)
- Figure 3.5 shows an attempted solution that fails: the queen in the rightmost column is attacked by the queen at the top left.
- There are two main kinds of formulation.
- An **incremental** formulation involves operators that *augment the state* description, starting with an empty state; for the 8-queens problem, this means that each action adds a queen to the state.
- A complete-state formulation starts with all 8 queens on the board and moves them around. In either case, the path cost is of no interest because only the final state counts.

- The first incremental formulation is the following:
- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked.

This formulation has $64 \cdot 63 \cdot 52 \cdot 49 \cdot 45 \cdot 41 \cdot 37 \cdot 33 \approx 1.8 \times 10^{14}$ possible sequences to investigate. Prohibit placing a queen in any square that is already attacked:

- **States:** All possible arrangements of n queens ($0 \leq n \leq 8$), one per column in the leftmost n columns, with no queen attacking another.
- **Actions:** Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

- This formulation reduces the 8-queens state space from 1.8×10^{14} to just 2,057, and solutions are easy to find.
- For 100 queens the reduction is from roughly 10400 states to about 1052 states a big improvement, but not enough to make the problem tractable.

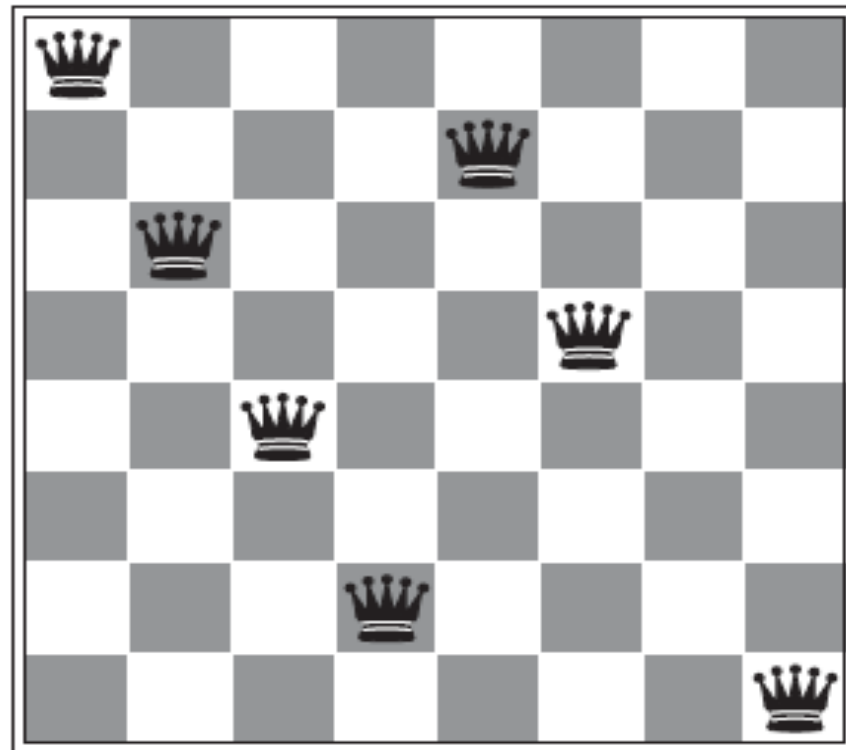


Figure 3.5 Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

Real-world problems- ROUTE-FINDING PROBLEM

- **Route-finding problem** is defined in terms of specified locations and transitions along links between them.
- Route-finding algorithms are used in a variety of applications. Some, such as Web sites and in-car systems that provide driving directions, are relatively straightforward extensions of the Romania example.
- Others, such as routing video streams in computer networks, military operations planning, and airline travel-planning systems, involve much more complex specifications.
- Consider the airline travel problems that must be solved by a travel-planning Web site:

- **States:** Each state obviously includes a location (e.g., an airport) and the current time.

Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and their status as domestic or international, the state must record extra information about these “historical” aspects.

- **Initial state:** This is specified by the user’s query.
- **Actions:** Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model:** The state resulting from taking a flight will have the flight’s destination as the current location and the flight’s arrival time as the current time.
- **Goal test:** Are we at the final destination specified by the user?
- **Path cost:** This depends on monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, and so on.